

Computational Extension of OEIS Sequence A029483: Two New Terms via Block Modular Concatenation in Base 14

Carlo Corti
Independent researcher
`carlo.corti@outlook.com`

June 2026

Abstract

OEIS sequence A029483 consists of the positive integers k such that k divides the left concatenation of $1, 2, \dots, k$, written in base 14. We report a segmented computation over the interval

$$[30000000001, 300000000000],$$

finding exactly two new terms in that interval:

$$44445214853, \quad 243987299933.$$

Together with the fourteen terms already present in the source OEIS record, these give a proposed b-file extension through $a(16)$. The computation does not construct the concatenated integer. For each candidate k , the base-14 left concatenation is evaluated modulo k by digit-length blocks, using arithmetic-geometric sums modulo k . An elementary filter removes all even candidates and all multiples of 7, leaving 115714285714 candidates for the full modular test out of 270000000000 raw integers. The production search was implemented as a single C17 program using pthreads and 64-bit candidates with `__uint128_t` modular products. The two new values, and the fourteen previously listed values, were independently certified by a Python checker based on a 3×3 matrix method, independent of the C program's recursive series-combination kernel. The negative part of the finite search result is based on the validated production kernel and recorded segment manifest; the independent checker certifies the reported positive values and known-term replay, not a second exhaustive scan of all non-hits.

2020 Mathematics Subject Classification. Primary 11Y55; Secondary 11A63, 11B83.

Key words and phrases. integer sequence, OEIS, concatenation, base 14, modular arithmetic, computational number theory, segmented search.

1 Introduction

OEIS sequence A029483 is defined as the sequence of positive integers k that divide the left concatenation of all integers $1, 2, \dots, k$, written in base 14 [1]. In the OEIS source snapshot used for this project, the known terms were

$$1, 13, 143, 169, 221, 403, 587, 11219, 178357, 222157, \\ 85762339, 1086336563, 8005332049, 15081597011,$$

and the entry recorded that no other terms occur below $3 \cdot 10^{10}$. The present computation extends the search above that bound to $3 \cdot 10^{11}$.

A029483 belongs to a broader OEIS family of concatenation-divisibility sequences in which the base and the direction of concatenation vary. That family context is useful provenance, but no transfer theorem between bases or directions is used here; this paper treats only the base-14 left-concatenation case.

The word “left” is important. The concatenation considered here is the base-14 integer obtained by successively prepending the current integer. Thus, for example, after processing 1, 2, 3, the represented base-14 digit string is the digit string for 3, followed by that for 2, followed by that for 1. Directly constructing this integer is impossible at the scale considered here: for k near $3 \cdot 10^{11}$, the concatenation has trillions of base-14 digits. The computation instead evaluates the concatenation modulo the candidate divisor k , independently for each candidate.

The main result of this paper is the following finite computational extension.

Theorem 1 (Validated production search result). *Subject to the recorded segment manifest and the correct execution of the validated production kernel described below, the only integers k in the interval*

$$[300000000001, 3000000000000],$$

satisfying the A029483 divisibility condition are

$$44445214853 \quad \text{and} \quad 243987299933.$$

Corollary 1. *Combining Theorem 1 with the source OEIS record used as input to this project, the proposed extension of the b-file for A029483 through the searched range has sixteen terms, with*

$$a(15) = 44445214853, \quad a(16) = 243987299933.$$

The rest of the paper explains the computation supporting Theorem 1. Section 2 fixes notation for left concatenation. Section 3 gives the block modular evaluation used by the production program. Section 4 describes the filter and segmented search, while Section 5 records the particular computational interpretation of this search. Section 6 records implementation and reproducibility details, and Section 7 records the independent certification and validation evidence. Section 8 summarizes the new terms and the proposed b-file extension.

Notation

For an integer $b \geq 2$ and $n \geq 1$, let

$$\ell_b(n) = 1 + \lfloor \log_b n \rfloor$$

be the number of base- b digits of n . Let

$$L_b(n) = \sum_{j=1}^n \ell_b(j), \quad L_b(0) = 0.$$

For later use, if $d \geq 1$, then

$$L_b(b^{d-1} - 1) = \sum_{i=1}^{d-1} i(b^i - b^{i-1}),$$

with the empty sum equal to 0. This is the total number of base- b digits contributed by the positive integers with fewer than d digits. All intervals of integers in this paper are inclusive unless explicitly stated otherwise.

2 Definition of the left concatenation

Definition 1. For $b \geq 2$ and $k \geq 1$, define the left concatenation

$$C_b(k) = \sum_{n=1}^k n b^{L_b(n-1)}.$$

The sequence considered here is

$$A029483 = \{k \geq 1 : C_{14}(k) \equiv 0 \pmod{k}\}.$$

This formula agrees with the OEIS Mathematica program in the local source snapshot, in which the base-14 digits of the current integer are prepended to the digit list already constructed. The term n is shifted left by exactly the number of base-14 digits already present, namely $L_b(n-1)$.

3 Block modular evaluation

The computation evaluates $C_{14}(k) \bmod k$ without constructing $C_{14}(k)$. The key observation is that the summation naturally splits into blocks on which all integers have the same base-14 digit length.

Proposition 1 (Digit-length block formula). *Let $b = 14$. Fix $k \geq 1$. For a digit length $d \geq 1$, set*

$$a = b^{d-1}, \quad u = \min(k, b^d - 1), \quad q = b^d,$$

and suppose $a \leq k$. Let $N = u - a + 1$ and $s = L_b(a - 1)$. The contribution to $C_b(k)$ from the block $a \leq n \leq u$ is

$$b^s \sum_{r=0}^{N-1} (a + r) q^r \pmod{k}.$$

Consequently,

$$C_b(k) \equiv \sum_{\substack{d \geq 1 \\ b^{d-1} \leq k}} b^{L_b(b^{d-1}-1)} \sum_{r=0}^{\min(k, b^d-1)-b^{d-1}} (b^{d-1} + r)(b^d)^r \pmod{k}.$$

Proof. For $n = a + r$ in the digit-length d block, the integers preceding n in the left-concatenation construction contribute

$$L_b(n-1) = L_b(a-1) + dr = s + dr$$

base- b digits. Hence the summand in $C_b(k)$ is

$$(a + r)b^{s+dr} = b^s(a + r)(b^d)^r.$$

Summing over $0 \leq r < N$ gives the block contribution. Summing the contributions over all digit lengths up to k gives the stated congruence. $\square$

For a fixed q , N , and modulus k , the inner sum can be written

$$aS_0(q, N) + S_1(q, N),$$

where

$$S_0(q, N) = \sum_{r=0}^{N-1} q^r, \quad S_1(q, N) = \sum_{r=0}^{N-1} r q^r.$$

The production C program computes the triple

$$(q^N, S_0(q, N), S_1(q, N)) \pmod{k}$$

by recursive series composition. The recursion splits a requested length N by halving it, or by removing a final odd term and then halving, and combines the resulting subsegments; it is not a linear scan through all N summands. In this computational discussion $q = b^d$, with its residue modulo k used in the modular arithmetic. The base cases are

$$(q^0, S_0(q, 0), S_1(q, 0)) = (1, 0, 0), \quad (q^1, S_0(q, 1), S_1(q, 1)) = (q, 1, 0).$$

If a left segment has length m and a right segment has length n , then

$$\begin{aligned} S_0(q, m+n) &= S_0(q, m) + q^m S_0(q, n), \\ S_1(q, m+n) &= S_1(q, m) + q^m (m S_0(q, n) + S_1(q, n)). \end{aligned}$$

Here m denotes the length of the left subsegment; in modular implementation it is reduced modulo k before being multiplied by a residue. These identities allow binary splitting in N , so each digit-length block is evaluated by a logarithmic number of modular products. Termination follows because each recursive call strictly reduces the requested segment length until one of the two displayed base cases is reached; correctness then follows by induction from the combination identities. This is the standard divide-and-conquer idea behind binary splitting [4]. For a fixed candidate k , the number of digit-length blocks is $O(\log k)$ and each block contributes $O(\log k)$ modular products, giving a qualitative $O((\log k)^2)$ modular-product cost per candidate for fixed base.

Remark 1. No division by $q-1$ is used. This avoids special cases when $q-1$ is not invertible modulo k .

4 Filtering and segmented search

The production search used one elementary but exact filter.

Lemma 1 (Base-factor exclusion). *For $k > 1$, if k is even or divisible by 7, then $k \notin A029483$.*

Proof. For $b = 14$, every summand $nb^{L_b(n-1)}$ with $n \geq 2$ is divisible by both 2 and 7, since $b = 14 = 2 \cdot 7$ and $L_b(n-1) > 0$. The $n = 1$ summand is 1. Therefore

$$C_{14}(k) \equiv 1 \pmod{2}, \quad C_{14}(k) \equiv 1 \pmod{7}.$$

If $k > 1$ were even and divided $C_{14}(k)$, then $C_{14}(k)$ would be congruent to 0 modulo 2, a contradiction. The argument modulo 7 is identical. $\square$

The search interval was

$$[30000000001, 300000000000],$$

containing 270000000000 raw integers. Lemma 1 leaves 115714285714 candidates for the full modular test. The production run was split into 23 consecutive segments, normally of 12000000000 raw integers each, with the last segment shortened to end exactly at 300000000000. The exact

segment record is stored in `results/A029483_official_run_segments.tsv` in the raw package [5]. The base-14 digit-length boundary $14^{10} = 289254654976$ lies inside the searched interval; candidates above this boundary are handled automatically by the same digit-length block formula. The survivor count can be checked directly from the six admissible residue classes modulo 14:

$$\sum_{r \in \{1,3,5,9,11,13\}} \left(\left\lfloor \frac{300000000000 - r}{14} \right\rfloor - \left\lfloor \frac{300000000000 - r}{14} \right\rfloor \right) = 115714285714.$$

Table 1 summarizes the aggregate run, and Table 2 lists the two segments in which new terms were found.

Raw integers	Survivors after filter	Segments	New hits
2700000000000	115714285714	23	2

Table 1: Aggregate production search over $[300000000001, 3000000000000]$.

Segment interval	Hit	Cumulative new hits
$[420000000001, 540000000000]$	44445214853	1
$[2340000000001, 2460000000000]$	243987299933	2

Table 2: The two production segments containing new A029483 terms.

The wall-clock time recorded across the segment manifest is 70941.341 seconds, or approximately 19.706 hours. This timing is reported as a property of the recorded production run, not as a hardware independent benchmark.

5 What is specific to this search

The present computation is best understood as a certified divisibility search, not as a statistical experiment. In contrast with prime-counting or square-value problems, A029483 does not come with an evident density model whose predictions can be checked numerically in a short paper. The divisor being tested is also the modulus in which the concatenation is evaluated, and the digit-length boundaries of the base-14 expansion enter the predicate directly. This makes the search highly deterministic but does not by itself suggest a credible random model for the location of the next term.

The one simple structural filter is nevertheless exact and unusually cheap: because the rightmost digit of the left concatenation is always the digit of 1, every candidate divisible by a prime factor of the base is impossible for $k > 1$. For base 14, this removes all even candidates and all multiples of 7. Over a complete residue period modulo 14, this leaves 3/7 of the integers; in the present finite interval the exact survivor count is 115714285714. The remaining work is not a sieve cascade but a repeated exact modular evaluation of $C_{14}(k)$ by digit-length blocks.

For this reason, the two new terms should not be read as evidence for a particular asymptotic frequency. Their significance is more modest and more concrete: they extend the certified table beyond the previous OEIS-recorded bound, and the segment manifest gives an explicit negative result for every other integer in the same interval.

6 Implementation and reproducibility

The production program is the C17 source file `src/a029483.c`, built with the accompanying `src/Makefile`. The implementation targets the C17 language standard [2] and uses POSIX threads [3]. Candidates and residues are stored in unsigned 64-bit integers. Modular products use the GCC/Clang extension `__uint128_t` as a double-width intermediate before reduction modulo the current candidate k . The program evaluates each candidate independently by the block formula of Proposition 1, so segmenting and resuming do not require carrying a concatenation state from one candidate modulus to the next. The largest modulus tested here is at most $3 \cdot 10^{11}$; before reduction, a product of two residues is therefore at most $(k - 1)^2$, and

$$(k - 1)^2 < k^2 \leq 9 \cdot 10^{22} < 2^{77},$$

well within the 128-bit intermediate range. In the binary-splitting combination for S_1 , the segment length m is first reduced modulo k ; consequently every multiplication passed to the double-width intermediate is still a product of two residues modulo k .

The production and validation host recorded for this project was a MinisForum UM790 Pro with an AMD Ryzen 9 7940HS processor, 8 physical cores and 16 hardware threads, and 64 GB of DDR5 memory. The local software environment used for the package validation was Linux Mint 22.3, Linux kernel 6.14.0-37-generic, and GCC 13.3.0. The Makefile mandatory flags were

```
-std=c17 -Wall -Wextra -Wshadow -Wconversion  
-Wdouble-promotion -Wformat=2 -pthread
```

with the optimized production build adding

```
-O3 -march=native -mtune=native -flto  
-funroll-loops -fomit-frame-pointer
```

and linking with `-flto -pthread -lm`.

The official command pattern was

```
src/a029483 \  
--resume results/a029483_checkpoint.txt \  
--end END \  
--block-size 120000000000 \  
--threads 16 \  
--out results/A029483_new_terms_u64.tsv \  
--checkpoint results/a029483_checkpoint.txt \  
--progress-every 1
```

with `END` set by segment. The `-block-size` argument is a raw-integer span before filtering, not the number of surviving candidates. The `-resume` option reads the next start from an existing checkpoint; `-checkpoint` names the checkpoint file to update after a completed segment. This command is shown as run from the extracted raw package root. The public raw package validation summary records that the following checks, shown from the parent project directory containing `GitHub/`, passed:

```
make -C GitHub/src -f Makefile clean  
make -C GitHub/src -f Makefile test  
make -C GitHub/src -f Makefile sanitize  
GitHub/scripts/certify_a029483.py --known --new  
GitHub/scripts/certify_a029483.py --file GitHub/results/A029483_new_terms_u64.tsv
```

The `test` target runs the internal self-test, a small-range regression over $[1, 250000]$ against the expected known terms in that interval, command line validation, and checkpoint error checks. The `sanitize` target reruns representative checks under ASan/UBSan. The two certifier invocations above replay both the previously known terms and the two new terms, and then verify the production hit file.

Segments are inclusive. Each invocation advances from the checkpoint file and sets a segment endpoint by `-end`; the checkpoint records the last completed endpoint, the next start, and the hit count, so a resumed run does not silently skip or duplicate an interval. A checkpoint is written only after the corresponding segment has completed successfully; it is written to a temporary file, flushed, synced, and then promoted to the checkpoint path. Candidate tests are independent. The parallel implementation assigns candidate ranges to worker threads and serializes shared hit, counter, and checkpoint updates. The validation record reports cross-thread determinism on $[1, 250000]$ for thread counts 1, 2, 4, 8, 16, as a regression check of the reported term set rather than a byte-for-byte claim about timing-bearing checkpoint files.

Before the production campaign, the final code had also passed a release build with strict warnings, the Makefile regression suite, ASan/UBSan, TSan, cross-thread determinism checks for thread counts 1, 2, 4, 8, 16, Valgrind self-test and smoke runs, scan-build, cppcheck triage, clang-tidy triage, and a throughput probe near $3 \cdot 10^{10}$. These records are summarized in `validation/validation_summary.md` and `validation/final_code_validation.md`.

7 Independent certification

The independent checker `scripts/certify_a029483.py` intentionally does not reuse the C program’s recursive arithmetic-geometric series combination. It uses the same digit-length block decomposition proved in Proposition 1, but evaluates each block through a linear recurrence represented by a 3×3 matrix transition. The independence is therefore kernel-level independence, not a second derivation of the block decomposition. The checker is a Python 3 script using only the standard library.

For a block beginning at a , with $q = b^d \bmod k$, define the state

$$v_i = \begin{pmatrix} q^i \\ iq^i \\ \sum_{r=0}^{i-1} (a+r)q^r \end{pmatrix} \pmod{k}.$$

Then

$$v_{i+1} = \begin{pmatrix} q & 0 & 0 \\ q & q & 0 \\ a & 1 & 1 \end{pmatrix} v_i \pmod{k}, \quad v_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Raising this 3×3 matrix to the block length gives the same block sum as Proposition 1, but through a different computational kernel: the third component advances by adding $(a+i)q^i$ to the accumulated block sum. This provides an independent certification path for reported values.

The validation summary records

```
k=44445214853: residue=0 direct_residue=n/a PASS
k=243987299933: residue=0 direct_residue=n/a PASS
```

for the two new values. The checker also certified the fourteen terms from the source OEIS record. In these large cases, `direct_residue=n/a` means that the checker did not attempt literal construction of the concatenated integer; that direct path is reserved for small values where the integer is feasible

to build. With its default setting, the checker compares the matrix residue against literal left-concatenation residues for $k \leq 5000$, which anchors the shared block decomposition on small cases. The reported residue is still computed modulo k by the independent matrix method above. During the campaign, a neighbor check around the second new term recorded

```
k=243987299932: residue=196861283305 direct_residue=n/a FAIL
k=243987299933: residue=0 direct_residue=n/a PASS
k=243987299934: residue=150270907739 direct_residue=n/a FAIL
```

where FAIL denotes the expected non-zero residue for a non-term, not a program failure. This is a local sanity check on the reported hit; the two neighboring residues are non-zero modulo their respective candidates.

8 Results

The new terms found by the production search are stored in `results/A029483_new_terms_u64.tsv`:

44445214853, 243987299933.

The proposed b-file extension stored in `data/b029483.txt` is shown in Table 3. The companion file `data/certified_terms.tsv` records source and certificate status for all sixteen rows; it is a certification table, not an OEIS a-file.

n	$a(n)$
1	1
2	13
3	143
4	169
5	221
6	403
7	587
8	11219
9	178357
10	222157
11	85762339
12	1086336563
13	8005332049
14	15081597011
15	44445214853
16	243987299933

Table 3: Proposed b-file extension for A029483 through the two terms found in this project.

Proof of Theorem 1. This is a computer-assisted finite proof. The official segment manifest partitions the interval $[30000000001, 300000000000]$ into 23 consecutive inclusive segments. The raw counts in the manifest sum to 270000000000, the size of the interval. By Lemma 1, every even integer and every multiple of 7 in the interval is impossible. The same manifest records 115714285714

surviving candidates after this exact filter, and these are the candidates evaluated by the block modular test of Proposition 1.

The production hit file `results/A029483_new_terms_u64.tsv` contains exactly the two rows 44445214853 and 243987299933. Both values lie in the interval and both are certified independently by the matrix-based checker described in Section 7. The negative claim depends on the integrity of the recorded segment manifest and the correct execution of the validated production kernel; the independent certificate confirms the reported positive hits and known-term replay, but it is not a second exhaustive scan of every non-hit. Subject to those recorded computational artifacts, the non-surviving candidates are excluded by Lemma 1, and every surviving candidate in the segment partition was evaluated by the production program. Hence the two certified values are exactly the A029483 terms found in the searched interval. $\square$

Proof of Corollary 1. The local OEIS source snapshot lists the first fourteen terms and records no other terms below $3 \cdot 10^{10}$. This earlier negative bound is taken as input from that OEIS snapshot and is not re-derived in the present search. The present computation begins at 300000000001, the first integer strictly above the previously recorded bound. Theorem 1 then accounts for the interval from 300000000001 through 300000000000, adding exactly the two terms shown in Table 3. The displayed indices $a(15)$ and $a(16)$ therefore depend on the local OEIS source snapshot, including its negative statement below $3 \cdot 10^{10}$. A live check of the OEIS entry on 2026-06-03 showed the same fourteen terms and the same stated bound. $\square$

9 Conclusion

The search reported here changes the known data for A029483 immediately above the previously recorded bound. In the interval from 300000000001 through 300000000000, exactly two terms were found:

$$44445214853, \quad 243987299933.$$

Consequently the proposed b-file extension has sixteen terms, with these two values occupying $a(15)$ and $a(16)$. The computational content of the paper is therefore a finite certified extension of the table, not a statistical or heuristic model for the occurrence of such divisors.

No density law for A029483 is proposed here, and no probabilistic calculation is used to support the result. This is a deliberate limitation rather than an omission: for this sequence the reliable contribution is the certified extension of the table and the explicit negative result over the searched interval. The exclusion of all other integers in that interval rests only on the exact base-factor filter, the exhaustive block modular evaluation of the surviving candidates, and the independent matrix-based certification of the reported terms. A final live OEIS check was performed on 2026-06-03 and confirmed that the source entry still listed the same fourteen terms and the same bound below $3 \cdot 10^{10}$.

Data availability

The code, validation records, segment manifest, production hit file, proposed b-file, and certified-term table are part of the local project package and are to be included in the project repository and archive listed in the References when those public artifacts are finalized. The principal artifacts are the C17 source and Makefile, the official segment TSV, the new-term TSV, the proposed b-file, the certified-term TSV, the independent Python checker, and the validation summaries. No additional dataset beyond these project artifacts is needed to reproduce the reported finite extension.

AI use disclosure

This work used large language models as research assistants during the workflow for the A029483 project. DeepSeek was used in the initial triage of the sequence and project feasibility. An OpenAI Codex system labelled GPT-5 by the working environment was used during the code-review, manuscript-drafting, and manuscript-revision stages. Additional LLM systems, identified as Claude, Gemini, Grok, Kimi, Qwen, and Z.ai, were used as non-author peer reviewers of draft manuscript text.

The AI-assisted work included mathematical analysis of the base-14 left-concatenation predicate, design and refinement of the digit-length block evaluation, C17 implementation support, code-review triage, validation planning, and drafting of this manuscript.

Mathematical analysis. The modular block formula, the arithmetic-geometric series presentation, and the elementary exclusion of even candidates and multiples of 7 were developed and written with AI assistance. The identities used in the paper are proved explicitly in the text and were checked against the final implementation and the independent certificate script before inclusion.

Algorithm and code. The production C17 search program and Makefile were produced through an AI-assisted development and review loop. Their correctness does not rest on that provenance: it rests on the regression suite, sanitizer and tooling checks, cross-thread determinism checks, known-term replay, segmented production manifest, and the independent Python matrix certificate described in this paper.

Manuscript. The manuscript was drafted with AI assistance and revised by the author. Every quantitative claim included here was checked against the local computational artifacts: the OEIS source snapshot, the segment manifest, the new-term file, the proposed b-file, the certified-term table, and the validation summaries.

The author takes full scientific responsibility for the mathematical statements, the algorithm, the implementation, the numerical results, and the conclusions of this paper.

References

- [1] OEIS Foundation Inc., *Sequence A029483: Numbers k that divide the (left) concatenation of all numbers $\leq k$ written in base 14*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A029483>. Sequence created by O. Gérard; further terms by L. Reeves, Jul. 09, 2001; edited and updated by L. Reeves, Apr. 12, 2002; Mathematica program by R. Price, Mar. 12, 2020; $a(11)$ contributed by M. Alekseyev, May 15, 2011; $a(12)$ – $a(14)$ contributed by J. Yuen, Jun. 04, 2024. Local project snapshot used as source material, 2026-05-29; live check performed 2026-06-03.
- [2] International Organization for Standardization, *ISO/IEC 9899:2018, Information technology – Programming languages – C*, International Organization for Standardization, Geneva, 2018.
- [3] IEEE and The Open Group, *The Open Group Base Specifications Issue 7, 2018 edition; IEEE Std 1003.1-2017*, 2018. Available at <https://pubs.opengroup.org/onlinepubs/9699919799/>.
- [4] R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge University Press, Cambridge, 2010.
- [5] C. Corti, *A029483: Computational search package for OEIS sequence A029483*, GitHub repository and Zenodo archive, 2026. Planned repository:

<https://github.com/carcorti/A029483>. DOI to be finalized after publication and
GitHub–Zenodo synchronization: 10.5281/zenodo.xxxxxxxx.